

Unit Testing in Flask

created by hejing@uic.edu.hk

2020/04/10

0x00 Preface

Testing your code is an important task when you are writing your program.

Suppose you are the chef making cakes for guests and you forget to put the sugar in it. What if you serverd it to the guests without tasting it.

To avoid such embarrassment, you should taste it before serving.

This tutorial we will focus on how to write test cases.

Prior to this tutorial if you want to develop a web-app using Flask then you should enjoy this wonderful journey.

There are many approaches to test your Flask application. Like:

UnitTest, Nose , Pytest.

For this tutorial, we will use the UnitTest package.

0x01 Basic usage

First, define a class and inherit unittest.TestCase

```
import unittest

class TestClass(unittest.TestCase):
    pass
```

Second, define the two test methods.

```
import unittest

class TestClass(unittest.TestCase):

    # The setUp() method to do some preparatory work, such as establishing a
    # database connection.
    # It is automatically invoked by the test framework before the use case is
    # executed.
    def setUp(self):
        pass
```

```
# The tearDown() method is used to do some cleanup, such as disconnecting the
database.
# It is automatically invoked by the test framework after the use case
execution completes, including failure.
def tearDown(self):
    pass
```

Finally, write down your test methods

```
import unittest

class TestClass(unittest.TestCase):

    # The setUp() method to do some preparatory work, such as establishing a
    database connection.
    # It is automatically invoked by the test framework before the use case is
    executed.
    def setUp(self):
        pass

    # The tearDown() method is used to do some cleanup, such as disconnecting the
    database.
    # It is automatically invoked by the test framework after the use case
    execution completes, including failure.
    def tearDown(self):
        pass

    # test method
    def test_some_api(self):
        pass
```

0x02 Setup environment

```
pip install flask
```

0x03 Example

We are going to write a function to simulate user login.

Step 1 : login.py

```
# login.py

## import libraries
from flask import Flask, request, jsonify
```

```
## create a Flask instance
app = Flask(__name__)

## create a login api
@app.route("/login", methods=["POST"])
def login():

    name = request.form.get("name")
    password = request.form.get("password")

    # check if name / password empty
    # "" 0 [] () {} None ==> False
    if not all([name, password]):
        # if any empty return error result
        return jsonify(code=1001, message='Parameter incomplete!')

    # on your project should check with DB
    # here jsut for case study
    if name == "admin" and password == "123321":
        return jsonify(code=0, message="Success!")

    else:
        return jsonify(code=1001, message="Not correct username or password!")

if __name__ == '__main__':
    ## debug parameter set as True to make changes without restarting the program
    app.run(debug=True)
```

Step 2 : Postman to test

Case 1: empty password

The screenshot shows a Postman interface for a POST request to `http://127.0.0.1:5000/login`. The request body is set to form-data. The parameters are:

KEY	VALUE	DESCRIPTION
☒ name	admin	
☒ password		

The response is shown in the bottom panel, which is set to JSON. The response body is:

```
{
  "code": 1001,
  "message": "Parameter incomplete!"
}
```

Case 2: Not correct password

POST http://127.0.0.1:5000/login Send Save

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings Cookies Code

none form-data x-www-form-urlencoded raw binary GraphQL

KEY	VALUE	DESCRIPTION
☒ name	admin	
☒ password	123456	
Key	Value	Description

Body Cookies Headers (4) Test Results Status: 200 OK Time: 8 ms Size: 215 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "code": 1001,
3   "message": "Not correct username or password!"
4 }
```

Case 3: Success

POST http://127.0.0.1:5000/login Send Save

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings Cookies Code

none form-data x-www-form-urlencoded raw binary GraphQL

KEY	VALUE	DESCRIPTION
☒ name	admin	
☒ password	123321	
Key	Value	Description

Body Cookies Headers (4) Test Results Status: 200 OK Time: 7 ms Size: 187 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "code": 0,
3   "message": "Success!"
4 }
```

Step3: Custom the unit test

```

import unittest
from login import app
import json

class TestLogin(unittest.TestCase):
    """ Define Test Case"""

    def setUp(self):
        """It is called before the specific test method is executed."""
        self.app = app

        # Activation test flag
        app.config['TESTING'] = True
        # Here, the test is performed using the test client provided by flask
        self.client = app.test_client()

    def test_empty_name_or_password(self):
```

```
        """Test empty name or password"""

        response = self.client.post("/login", data={})
        # response data
        resp_json = response.data

        # convert to json
        resp_dict = json.loads(resp_json)

        # assert if the code in the dict
        self.assertIn("code", resp_dict)

        # compare the code value 1001
        code = resp_dict.get("code")
        self.assertEqual(code, 1001)

        # Test only recieve name
        response = self.client.post("/login", data={"name": "admin"})

        # response data
        resp_json = response.data

        # convert to json
        resp_dict = json.loads(resp_json)

        # use assert to verify
        self.assertIn("code", resp_dict)

        # compare the code value 1001
        code = resp_dict.get("code")
        self.assertEqual(code, 1001)

        # return message
        msg = resp_dict.get('message')
        self.assertEqual(msg, "Parameter incomplete!")

    def test_wrong_name_password(self):
        """Test wrong name or password"""

        response = self.client.post("/login", data={"name": "admin", "password":
"123456"})

        # response data
        resp_json = response.data

        # convert to json
        resp_dict = json.loads(resp_json)

        # use assert to verify
        self.assertIn("code", resp_dict)

        # compare the code value 1001
        code = resp_dict.get("code")
        self.assertEqual(code, 1001)
```

```

# return message
msg = resp_dict.get('message')
self.assertEqual(msg, "Not correct username or password!")

def test_correct_name_password(self):

    response = self.client.post("/login", data={"name": "admin", "password":
"123321"})

    # response data
    resp_json = response.data

    # convert to json
    resp_dict = json.loads(resp_json)

    # use assert to verify
    self.assertIn("code", resp_dict)

    # compare the code value 0
    code = resp_dict.get("code")
    self.assertEqual(code, 0)

    # return message
    msg = resp_dict.get('message')
    self.assertEqual(msg, "Success!")

if __name__ == '__main__':
    unittest.main()

```

Execute the test as follows:

The screenshot shows a code editor with two files: `test_login.py` and `login.py`. The `test_login.py` file contains the test code shown in the previous block. The `login.py` file contains the Flask application code. The terminal window at the bottom shows the output of running the tests:

```

(workshop3_venv)
77413@DESKTOP-KQ727C3 MINGW64 /d/Data/UIC/WorkshopIII/Workshop/2020 (master)
$ E:/Workshop/py_project/venv/workshop3_venv/Scripts/python.exe d:/Data/UIC/WorkshopIII/Workshop/2020/UnitTest/code/test_login.py
...
Ran 3 tests in 0.007s
OK
(workshop3_venv)
77413@DESKTOP-KQ727C3 MINGW64 /d/Data/UIC/WorkshopIII/Workshop/2020 (master)
$

```

This is the basic processes for Flask framework unit tests.